1. Лабораторная работа №1

Многопоточное программирование

Цель работы: знакомство с пространствами имен **System.Threading**.

Задача работы: научиться запускать методы в отдельных потоках, использовать механизмы синхронизации для управления работой потоков.

Время выполнения работы: 4 часа (2 занятия)

Результат выполнения работы: программа, обеспечивающая функционал согласно заданию.

Классы создавать в отдельном проекте – библиотека классов .NET.

1.1. Задание .

1) Создать класс, содержащий метод вычисления интеграла функции y=sin(x) на участке от 0 до 1 (использовать метод прямоугольников). Для итерации использовать шаг 0,00000001. Для увеличения времени выполнения вычисления на каждой итерации введите задержку в виде цикла из 100000 формальных вычислений (например, умножения двух чисел)

2) Предусмотреть в методе получение времени, затраченного на выполнение метода (использовать методы класса StopWatch). Результат выполнения передать с помощью события.

3) Предусмотреть возможность вывода информации о прогрессе выполнения метода (**использовать механизм событий, без применения интерфейса IProgress**).

4) В классе **Program**:

– Запустить вычисление интеграла в отдельном потоке

– Выводить в консоль прогресс выполнения метода вычисления в виде:



– После завершения вычисления вывести:

Поток xxxxxxxxxx: Завершен с результатом: XXXX

– Запустить два экземпляра метода в разных потоках, предварительно установив приоритеты для одного потока Highest, а для второго Lowest. По завершении вычислений вывести результат и время выполнения для каждого потока.

Рекомендация к заданию: поскольку процессы выполняются быстро, время выводить не в ms, а в ticks

5) Доработать код функции вычисления интеграла так, чтобы при запуске функции в нескольких потоках, выполнялся только один поток. Проверьте результат, запустив сразу 5 потоков.

6) Доработать код функции вычисления интеграла так, чтобы при запуске функции в нескольких потоках, выполнялся только заданное количество потоков (1, 2, 3 …), а остальные переходили в режим ожидания. Проверьте результат, запустив сразу 5 потоков.

1.2. Вопросы для самопроверки

1. Может ли существовать процесс без потоков?
2. Может ли в одном процессе выполняться несколько потоков?
3. Верно ли утверждение: все потоки одного процесса работают в одном адресном пространстве?
4. Что такое «Вытесняющий алгоритм диспетчеризации потоков»?
5. Какие три состояния потока вы знаете?
6. В каком диапазоне можно менять приоритет созданного потока?
7. Как запустить поток в коде C#?
8. Как получить состояние потока?
9. Для чего используется метод Join класса Thread?
10. Что такое пул потоков?
11. Что такое «гонки» применительно к потокам?
12. Что такое синхронизация потоков?
13. Чем отличается Mutex от Semaphore?
14. Можно ли с помощью конструкции «lock» синхронизировать процессы?
15. Можно ли с помощью Semaphore синхронизировать процессы?

16. Для чего используется класс Interlocked?

17. Чем отличается работа таймеров System.Threading.Timer и System.Timers.Timer?

2. Лабораторная работа №2

TPL

Цель работы: знакомство с пространством имен system.Threading.Tasks

Задача работы: научиться запускать методы в отдельных потоках, использовать механизмы синхронизации для управления работой потоков.

Время выполнения работы: 4 часа

Результат выполнения работы: программа, обеспечивающая функционал согласно заданию.

Классы создавать в отдельном проекте – библиотека классов .NET.

2.1. Задание

Описать **один** класс предметной области согласно варианту. Класс должен содержать свойства:

- Id (уникальный номер),
- название объекта,
- свойство, необходимое для подсчета статистической информации

Создать класс **StreamService<T>**, в котором описать три метода:

1. `public async Task WriteToStreamAsync(Stream stream, IEnumerable<T> data, IProgress<string> progress)` – записывает коллекцию ***data*** в поток ***stream***; **ОБЯЗАТЕЛЬНО**:установите задержку, чтобы запись выполнялась медленно, в течение 3-5 секунд.
2. метод `public async Task CopyFromStreamAsync(Stream stream, string filename, IProgress<string> progress)` - копирует информацию из потока ***stream*** в файл с именем ***fileName***;
3. метод `public async Task<int> GetStatisticsAsync(string fileName, Func<T, bool> filter)` считывает объекты типа ***T*** из файла с именем ***filename*** и возвращает количество объектов, удовлетворяющих условию ***filter***.

Использовать **асинхронные** методы чтения/записи в поток/файл

Предусмотреть **синхронизацию с помощью объектов синхронизации** методов 1 и 2 (копирование должно выполняться только после окончания записи)

В методах 1 и 2 предусмотреть оповещение о начале и конце записи/чтения в/из потока ***stream*** с указание номера потока выполнения (`Thread.CurrentThread.ManagedThreadId`). Соответствующее сообщение должно быть выведено в консоль в классе Program (использовать IProgress)

В классе Program:

1. Сделать метод Main асинхронным (`static async Task Main(string[] args)`

2. Создать коллекцию из 1000 объектов согласно индивидуальному заданию.
3. Вывести в консоль номер потока выполнения и сообщение о начале работы;
4. Синхронно (без использования ***await***) запустить методы 1 и 2 класса StreamService (для гарантирования последовательности запуска потоков установите задержку 100-200 мс между запусками методов). В качестве данных передавайте коллекцию из п.2. В качестве параметра stream использовать MemoryStream. Методы 1 и 2 должны использовать **один и тот же** экземпляр ***stream***.
5. Вывести в консоль номер потока выполнения и сообщение о том, что потоки 1 и 2 запущены;
6. **Ожидать** завершениия выполнения методов 1 и 2;
7. **Асинхронно** получить статистические данные (метод `GetStatisticsAsync`);
8. Полученные статистические данные вывести в консоль
9. Модифицировать методы 1 и 2 для генерирования событий об этапах выполнения задания – вход в метод, начало записи/чтения, завершение записи/чтения (использовать IProgress). В событии передавать Id потока и выполняемое действие. В классе Program подписаться на событие и вывести в консоль информацию в виде:

Поток xxxxxxxxxx: yyyy

Где: xxx – id потока, yyy – информация об этапе выполнения задачи

Рекомендации к заданию 2:

1000 объектов можно создать в цикле, присваивая формальные значения свойств класса в виде $«[Имя свойства] {i}». Можно также использовать генератор случайных чисел.

Для генерирования случайных строковых данных можно использовать NuGet пакет LoremNET (https://www.nuget.org/packages/LoremNET/2.0.0 , https://github.com/trichards57/Lorem.Universal.NET/blob/master/readme.md)

2.2. Варианты заданий

1. **Предметная область – недвижимость**. Статистическая информация – количество домов, в которых количество жильцов больше 100;
2. **Предметная область – компьютеры.** Статистическая информация – количество компьютеров определенной марки
3. **Предметная область – пассажиры**. Статистическая информация – количество пассажиров, у которых есть багаж

4. **Предметная область – сотрудники предприятия**. Статистическая информация – количество сотрудников старше 35 лет

5. **Предметная область – автопарк**. Статистическая информация – количество автомобилей, у которых техосмотр должен проводиться в текущем году

6. **Предметная область – агентство по трудоустройству**. Статистическая информация – количество кандидатов на работу определенного профиля

7. **Предметная область – автосалон**. Статистическая информация – количество автомобилей с объемом двигателя более 2 литров

8. **Предметная область – художественная галерея**. Статистическая информация – количество работ определенного мастера

9. **Предметная область – пассажиры.** Статистическая информация – количество пассажиров, у которых есть багаж

10. **Предметная область – продукты питания**. Статистическая информация – количество продуктов, у которых закончился срок годности

11. **Предметная область – музыка**. Статистическая информация – количество песен определенного исполнителя

12. **Предметная область – студенты**. Статистическая информация – количество студентов, у которых средний балл больше 9

13. **Предметная область – биология**. Статистическая информация – количество существ, которые умеют летать

14. **Предметная область – обучающие курсы**. Статистическая информация – количество курсов, на которых количество слушателей больше 10

15. **Предметная область – игрушки**. Статистическая информация – количество игрушек для детей от 6 лет

16. **Предметная область – клиенты банка**. Статистическая информация – количество клиентов, открывших счет в текущем году

17. **Предметная область – багаж.** Статистическая информация – количество багажа, у которого вес превышает 20 кг.

18. **Предметная область – пациенты больницы.** Статистическая информация – количество пациентов с определенным диагнозом

19. **Предметная область –соревнования по стрельбе**. Статистическая информация – количество спортсменов, у которых сумма баллов больше 80

20. **Предметная область – персонажи игры**. Статистическая информация – количество персонажей, которые владеют определенным оружием.

2.3. Вопросы для самопроверки

1. Как запустить задачу (Task), подкрепленную потоком?
2. В чем отличие синхронных операций от асинхронных?
3. Для чего используется ключевое слово «async»?
4. Для чего используется ключевое слово «await»?
5. Как реализовать ожидание завершения выполнения нескольких задач?
6. Как указать, что делать по завершении задачи (цепочка задач, Continuation Task)
7. Как отменить запущенную задачу?
8. Как использовать класс Progress для информирования о процессе выполнения задачи?
9. Для чего используется класс Parallel из пространства имен System.Threading.Tasks?
10. Как параллельно выполнить запрос к коллекции?
11. Приведите примеры потокобезопасных коллекций
12. Чем обычная коллекция отличается от потокобезопасной коллекции?